

Програмиране на C

Лекция 11

Указатели и функции

Стек вызовов

При вызове функции программа должна запомнить:

- параметры функции;
- локальные переменные;
- место, куда нужно вернуться после завершения.

Для этого используется **стек вызовов**.

Пример вызовов

```
void bar(void)
{
}

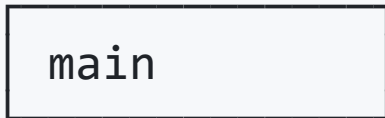
void foo(void)
{
    bar();
}

int main(void)
{
    foo();

    return 0;
}
```

Как меняется стек?

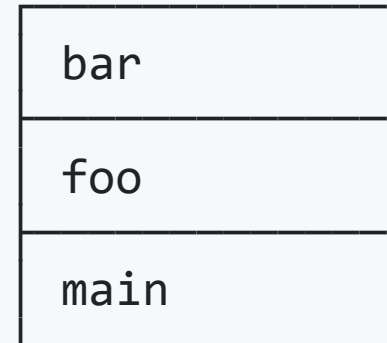
начало программы



main вызывает foo

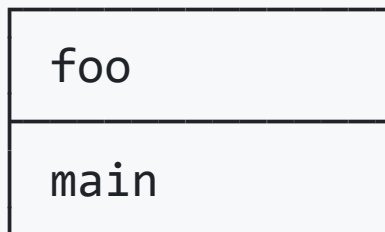


foo вызывает bar

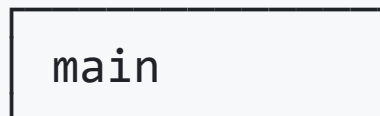


Возврат из функций

bar завершилась



foo завершилась



main завершилась

стек пуст

Рекурсия

Рекурсия — это ситуация, когда функция вызывает саму себя.

```
void countdown(int n)
{
    if (n == 0)
        return;

    printf("%d\n", n);

    countdown(n - 1);
}
```

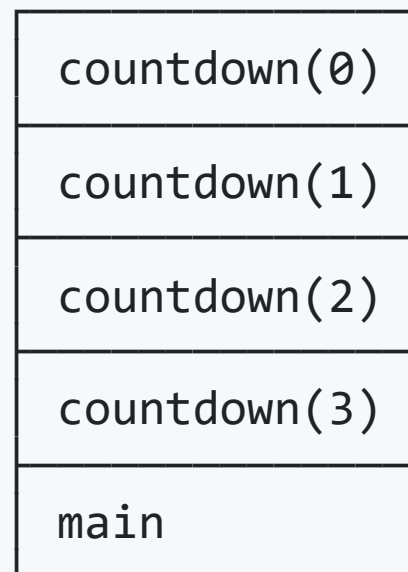
Рекурсия и стек

```
void countdown(int n)
{
    if (n == 0)
        return;

    printf("%d\n", n);

    countdown(n - 1);
}
```

```
countdown(3);
```



База рекурсии

```
if (n == 0)  
    return;
```

База рекурсии — условие, при котором рекурсивные вызовы прекращаются.

Без базы рекурсия не остановится.

Факториал

```
int factorial(int n)
{
    if (n == 0)
        return 1;

    return n * factorial(n - 1);
}
```

factorial(4)

4 * factorial(3)

4 * 3 * factorial(2)

4 * 3 * 2 * factorial(1)

4 * 3 * 2 * 1 * factorial(0)

factorial(0) = 1

factorial(1) = 1 * 1 = 1

factorial(2) = 2 * 1 = 2

factorial(3) = 3 * 2 = 6

factorial(4) = 4 * 6 = 24

Переполнение стека

```
void f(void)
{
    f();
}
```



Если рекурсия не останавливается, стек переполняется.

Если рекурсия слишком глубокая, стек переполняется.

Адрес функции

У переменной есть адрес.

У функции тоже есть адрес.

```
int add(int a, int b)
{
    return a + b;
}
```

Имя функции можно использовать как указатель на её код.

Указатель на функцию

```
int (*operation)(int, int);
```

`operation` — указатель на функцию, которая принимает два `int` и возвращает `int`.

Присваивание функции указателю

```
int add(int a, int b)
{
    return a + b;
}

int (*operation)(int, int);

operation = add;
```

Теперь через `operation` можно вызвать функцию `add`.

Вызов через указатель

```
int result = operation(2, 3);
```

Эквивалентная запись:

```
int result = (*operation)(2, 3);
```

5

Несколько функций одного типа

```
int add(int a, int b)
{
    return a + b;
}
```

```
int sub(int a, int b)
{
    return a - b;
}
```

```
int mul(int a, int b)
{
    return a * b;
}
```

Один указатель — разные функции

```
int (*operation)(int, int);

operation = add;
printf("%d\n", operation(6, 2));

operation = sub;
printf("%d\n", operation(6, 2));

operation = mul;
printf("%d\n", operation(6, 2));
```

```
8
4
12
```

typedef для указателя на функцию

```
typedef int (*operation_t)(int, int);
```

Теперь можно писать проще:

```
operation_t operation;  
  
operation = add;
```

typedef **делает сложный тип читаемым.**

Массив указателей на функции

```
operation_t operations[] = {  
    add,  
    sub,  
    mul  
};
```

Вызов:

```
int result = operations[2](6, 2);
```

12

Таблица операций

```
operation_t operations[] = {  
    add,    // 0  
    sub,    // 1  
    mul     // 2  
};  
  
int code = 0;  
  
printf("%d\n", operations[code](6, 2));
```

Массив указателей на функции позволяет выбирать действие по номеру.

Функция как параметр

Функцию можно передать в другую функцию через указатель.

```
void apply(int a, int b, operation_t operation)
{
    printf("%d\n", operation(a, b));
}
```

Вызов:

```
apply(6, 2, add);
apply(6, 2, mul);
```

Зачем это нужно?

Указатели на функции позволяют:

- выбирать действие во время выполнения;
- передавать алгоритм как параметр;
- строить таблицы обработчиков;
- избегать длинных цепочек `if` и `switch` .

Функция `main`

До сих пор мы использовали функцию

```
int main(void)
{
    ...
}
```

Однако у `main` есть и другая форма записи.

Аргументы командной строки

При запуске программы можно передать ей параметры.

Например:

```
./myprog file.txt 123 hello
```

или в Windows:

```
myprog.exe file.txt 123 hello
```

Эти параметры передаются функции `main`.

Сигнатура main

```
int main(int argc, char *argv[])
```

где

- `argc` — количество аргументов командной строки;
- `argv` — массив строк.

Каждый элемент массива `argv` содержит один аргумент командной строки.

Что находится в argv и argc?

Если программа запущена так:

```
./myprog file.txt 123 hello
```

то

индекс	значение
argc	4
argv[0]	"./myprog"
argv[1]	"file.txt"
argv[2]	"123"
argv[3]	"hello"

argv — это массив

argv



"./myprog"
"file.txt"
"123"
"hello"

Вывод аргументов

```
int main(int argc, char *argv[])
{
    for (int i = 1; i < argc; i++)
        printf("%s\n", argv[i]);

    return 0;
}
```

Запуск

```
./myprog file.txt 123 hello
```

Выведет

```
file.txt
123
hello
```

Почему цикл начинается с 1?

Первый элемент массива

```
argv[0]
```

содержит имя программы.

Поэтому пользовательские аргументы начинаются с

```
argv[1]
```

Переменное число аргументов

Большинство функций принимает фиксированное число параметров.

Например:

```
int max(int a, int b);
```

Но некоторые функции могут принимать произвольное количество аргументов.

Например:

```
printf(...)
```

printf

```
printf("%d\n", 10);  
  
printf("%d %d\n", 10, 20);  
  
printf("%d %d %d\n", 10, 20, 30);
```

Количество аргументов зависит от строки формата.

Объявление функции

Для функций с переменным числом параметров используется многоточие.

```
int printf(const char *format, ...);
```

... означает, что после обязательных параметров могут следовать дополнительные аргументы.

Работа с переменным числом аргументов

Для этого используется заголовочный файл

```
#include <stdarg.h>
```

Основные макросы:

- `va_list`
- `va_start`
- `va_arg`
- `va_end`

Пример

```
int sum(int count, ...)
{
    va_list args;

    va_start(args, count);

    int s = 0;

    for (int i = 0; i < count; i++)
        s += va_arg(args, int);

    va_end(args);

    return s;
}
```

Использование

```
printf("%d\n", sum(3, 10, 20, 30));  
printf("%d\n", sum(5, 1, 2, 3, 4, 5));
```

```
60  
15
```

Ограничения

Функция

```
sum(...)
```

не знает самостоятельно,

- сколько аргументов передано;
- какого они типа.

Поэтому эта информация должна передаваться отдельно.

Например: `sum(5, ...)` или `printf("%d %f", ...)`